A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

Kokil Vakratund Prakash (75)

Under the Guidance of

Prof. Tejal. H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]



BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Kokil Vakratund Prakash (75),** on **"Implementation of Snake and Ladder [BFS]" is done and submitted during 2026–2**027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal. H. Patil
(Guide)

Dr. Lalit. V. Patil
(Head of Computer Engg. Department)

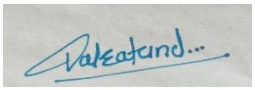
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal. H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit. V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Kokil Vakratund Prakash
Student Name



ABSTRACT

The mini-project “Implementation of Snake and Ladder Using Breadth-First Search (BFS)” aims to demonstrate the application of graph traversal algorithms in solving a classic board game. Snake and Ladder consists of a board with numbered squares, where players move according to dice rolls. Snakes move players backward, while ladders help them advance to higher-numbered squares. Finding the shortest path from the starting square to the destination can be represented as a graph-search problem.

In this project, the board is modelled as a graph in which each square represents a vertex and each possible dice roll represents an edge. The positions of snakes and ladders are considered while determining the next position. Breadth-First Search (BFS) is used to explore the possible moves level by level and identify the minimum number of dice rolls required to reach the final square, assuming each legal dice outcome has equal cost.

The project helps in understanding graph representation, queue operations, visited-node tracking, and shortest-path algorithms. It demonstrates how BFS can solve a real-world-style problem systematically and efficiently. This implementation also provides a foundation for extending the game with features such as interactive gameplay, different board configurations, and visual representation of the shortest path.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

1 Introduction 1
   a) Overview
   b) Problem Statement
   c) Objective

2 Literature Survey 2

3 UML Diagram 3

4 System Requirement 4

5 System Design 5
   a) Existing system
   b) Proposed system
   c) Working Diagram
   d) System code

6 Result 8

7 Application 9

8 Future Scope 10

9 Conclusion 11

10 References 12

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	04
Fig 5.2	System Code	07
Fig 5.3	Result	12

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

1. INTRODUCTION

1.1 Overview :

Snake and Ladder is a traditional board game in which players move across numbered squares based on the outcome of a dice roll. The board contains snakes that move players backward and ladders that help them advance to higher positions. Although the game is simple to play, finding the minimum number of dice rolls required to reach the final square can be treated as a graph traversal problem.

This mini-project implements the Snake and Ladder game using the Breadth-First Search (BFS) algorithm. BFS is a graph traversal technique that explores nodes level by level using a queue. In this project, each square on the board is represented as a vertex, and the possible moves determined by dice rolls are represented as edges. The positions of snakes and ladders are considered when calculating the next position of a player.

The BFS algorithm explores all possible moves systematically and determines the shortest path from the starting square to the destination in terms of the minimum number of dice rolls. Since each dice roll is treated as one move, BFS is suitable for finding the shortest path when all moves have equal cost. This project demonstrates the practical application of graph theory and search algorithms in solving a familiar board game.

1.2 Objectives :

The main objective of this mini-project is to implement the Snake and Ladder game using the Breadth-First Search algorithm. The specific objectives are:

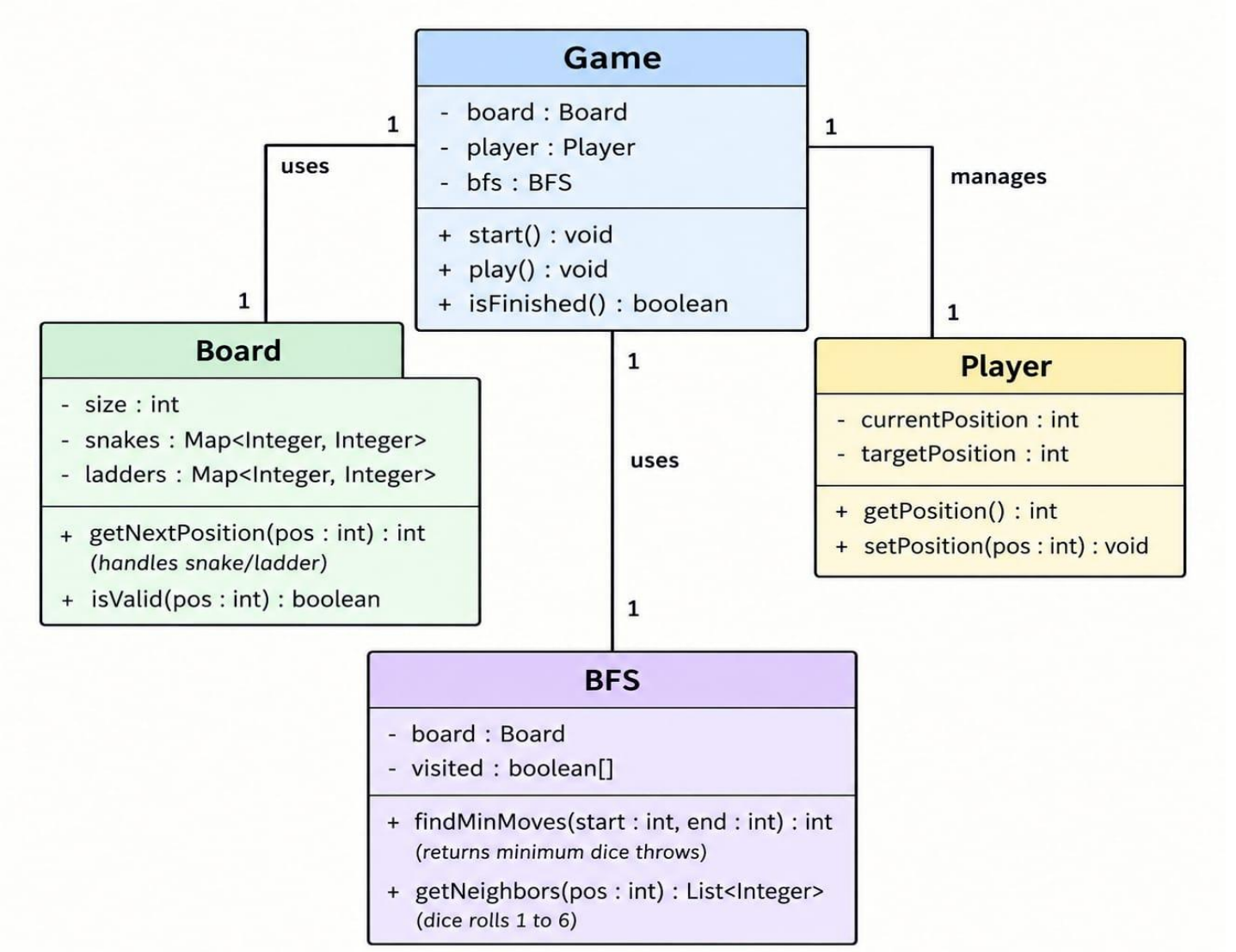
- To understand the concepts of graph representation and graph traversal.
- To implement the BFS algorithm using a queue.
- To represent the board squares as vertices and possible dice moves as edges.
- To consider the effects of snakes and ladders while calculating possible moves.
- To determine the minimum number of dice rolls required to reach the destination.
- To improve understanding of shortest-path algorithms and their practical applications.
- To develop a simple and understandable program that demonstrates the use of BFS in solving a board game problem.

2. LITERATURE SURVEY

Various researchers have studied airline reservation systems to improve booking operations, information management, and customer convenience. Research on sorting algorithms has also explored methods for understanding and applying these algorithms effectively. The following studies provide a foundation for the development of the proposed Flight Reservation System using searching and sorting techniques.

Year	Paper Name	Author	Conclusion
2025	Research on Strategy Optimization for the Snakes and Ladders Game Based on Graph Theory and Markov Chains	Yaowei Zhang	The study uses BFS to find the shortest path and minimum moves in Snake and Ladder, with additional algorithms for strategy optimization.
2025	A Review of Graph Traversal Algorithms: Techniques and Applications in Network Analysis	Chreesk Sabah M. Ali and Ibrahim Mahmood Ibrahim	The study reviews BFS and DFS and explains their uses, advantages, and limitations in graph traversal and network analysis.
2025	Comparing Traversal Strategies: Depth-First Search vs. Breadth-First Search in Complex Networks	Ayad Zedo Ismaeel and Ibarhim M. I. Zebari	The study compares BFS and DFS and highlights BFS as a suitable approach for finding shortest paths in unweighted graphs.
2025	Optimizing the 15 Puzzle with AI: Comparative Analysis of Breadth-First and Depth-First Search Algorithms	Iryna Ivanochko, Michal Greguš,Silvia Treľová.	The study applies BFS and DFS to a puzzle-solving problem and compares their performance, demonstrating how search algorithms can solve state-based problems.

3. UML DIAGRAM



UML Class Diagram – Snake and Ladder (BFS)

4. SYSTEM REQUIREMENT

4.1 Hardware Requirements

The hardware requirements for implementing the Snake and Ladder game using the Breadth-First Search (BFS) algorithm are as follows:

- Processor: Intel Dual-Core processor or equivalent
- RAM: Minimum 2 GB
- Hard Disk: At least 100 MB of free storage space
- Input Devices: Keyboard and mouse
- Output Device: Monitor or display screen

4.2 Software Requirements

The software requirements for developing and executing the mini-project are as follows:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python, C++, Java, or C
- Code Editor/IDE: Visual Studio Code, PyCharm, Code::Blocks, or any suitable editor
- Compiler/Interpreter: A suitable compiler or interpreter for the selected programming language
- Algorithm: Breadth-First Search (BFS)
- Data Structures: Graph, queue, and visited array or set
- Testing Environment: Terminal, command prompt, or IDE console

Note: These are general requirements for a basic implementation. The final software requirements should match the programming language and tools used in the actual project**.**

5. SYSTEM DESIGN

5.1 Existing System

In the existing system, Snake and Ladder is commonly played as a traditional board game in which players move according to dice rolls. Snakes move players to lower-numbered squares, while ladders help them reach higher-numbered squares. Players continue rolling the dice until they reach the final square.

However, the traditional game does not explicitly calculate the shortest path or the minimum number of dice rolls required to reach the destination. Finding the optimal path manually can be difficult when multiple possible moves and several snakes and ladders are present. Therefore, an algorithmic approach can help analyze possible moves and determine an efficient path.

5.2 Proposed System

The proposed system implements the Snake and Ladder game using the Breadth-First Search (BFS) algorithm. The board is represented as a graph, where each square is a vertex and each possible dice roll represents an edge. Snakes and ladders are incorporated into the movement rules.

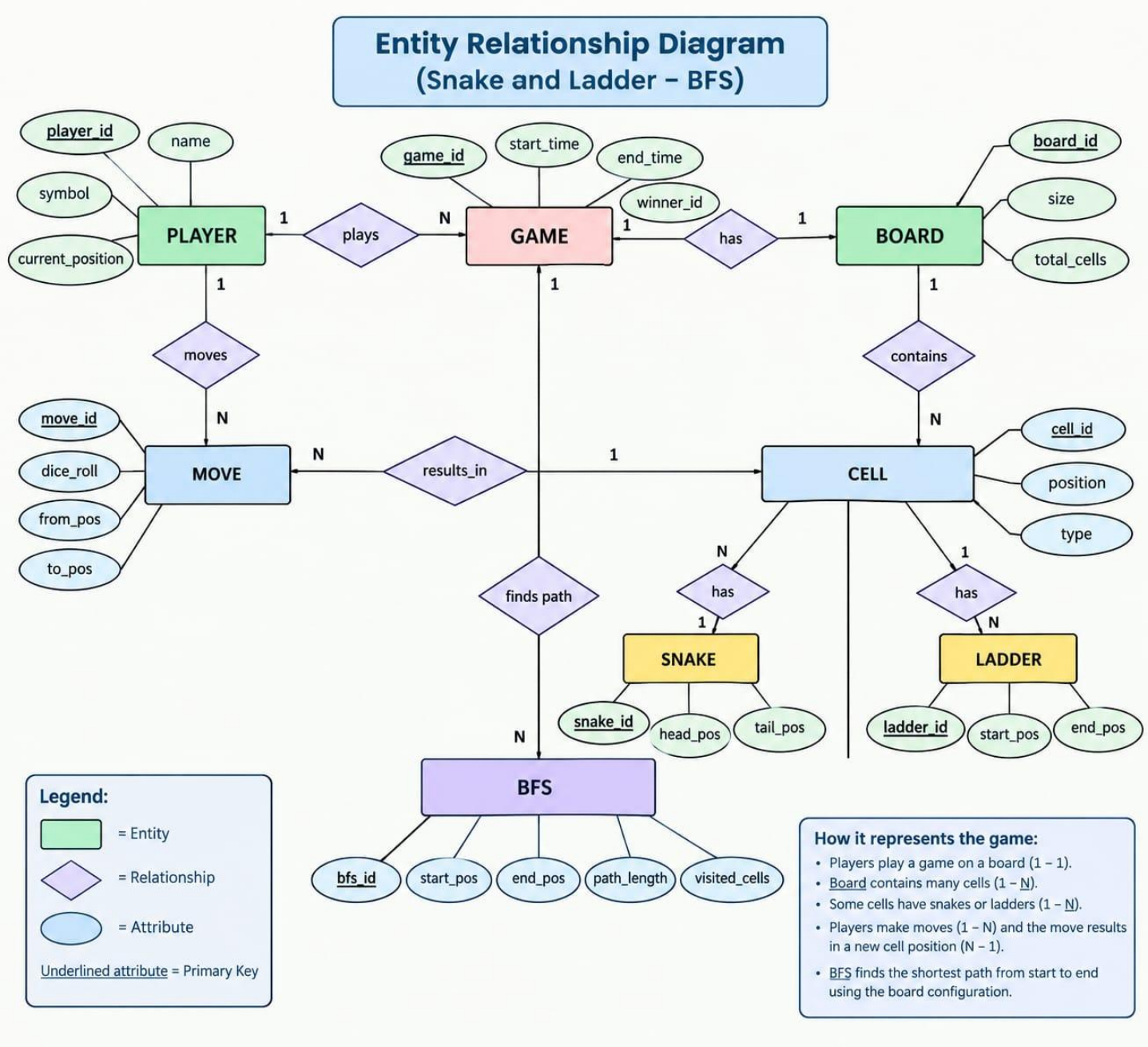
The BFS algorithm explores possible positions level by level using a queue and identifies the minimum number of dice rolls needed to reach the final square.

The main features of the proposed system are:

- Graph Representation: Represents board squares and possible moves as a graph.
- BFS Algorithm: Explores possible moves systematically to find the shortest path.
- Snake and Ladder Handling: Updates the player's position according to snakes and ladders.
- Minimum Moves Calculation: Determines the minimum number of dice rolls required to reach the destination.
- Visited Tracking: Prevents unnecessary repeated exploration of board positions.
- Efficient Solution: Provides a systematic approach to solving the game using graph traversal.

The proposed system demonstrates the practical application of BFS in shortest-path problems and improves understanding of graph theory, queues, and search algorithms.

Entity Relationship Diagram:



SYSTEM CODE

In [1]:

```
from collections import deque
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

SNAKES = {
    17: 7, 54: 34, 62: 19, 64: 60,
    87: 24, 93: 73, 95: 75, 99: 78
}

LADDERS = {
    4: 14, 9: 31, 20: 38, 28: 84,
    40: 59, 51: 67, 63: 81, 71: 91
}

JUMPS = {**SNAKES, **LADDERS}


def bfs_shortest_path(start=1, target=100):
    if not (1 <= start <= 100 and 1 <= target <= 100):
        raise ValueError("Squares must be between 1 and 100.")

    if start == target:
        return [start], [], 1

    queue = deque([start])
    parent = {start: None}

    while queue:
        current = queue.popleft()

        for dice in range(1, 7):
            landing = current + dice

            if landing > target:
                continue

            destination = JUMPS.get(landing, landing)

            if destination in parent:
                continue

            parent[destination] = (current, dice, landing)

            if destination == target:
                queue.clear()
                break

            queue.append(destination)

    if target not in parent:
        return None, [], len(parent)

    path = []
    actions = []
```

```
    square = target

    while square != start:
        path.append(square)
        previous, dice, landing = parent[square]

        actions.append({
            "from": previous,
            "dice": dice,
            "landing": landing,
            "to": square
        })

        square = previous

    path.append(start)
    path.reverse()
    actions.reverse()

    return path, actions, len(parent)


def square_coordinates(square):
    row = (square - 1) // 10
    col = (square - 1) % 10

    if row % 2 == 1:
        col = 9 - col

    return col, row


def draw_board(path=None):
    path = path or []
    path_set = set(path)

    fig, ax = plt.subplots(figsize=(12, 10))

    for square in range(1, 101):
        x, y = square_coordinates(square)

        if square == 1:
            color = "#A7F3D0"
        elif square == 100:
            color = "#FDE68A"
        elif square in path_set:
            color = "#93C5FD"
        elif square in SNAKES:
            color = "#FECACA"
        elif square in LADDERS:
            color = "#BBF7D0"
        else:
            color = "#F8FAFC" if (square + square // 10) % 2 else "#E2E8F0"

        ax.add_patch(Rectangle(
            (x, y), 1, 1,
```

```
            facecolor=color,
            edgecolor="#94A3B8",
            linewidth=0.8
        ))

        ax.text(
            x + 0.5, y + 0.64, str(square),
            ha="center", va="center",
            fontsize=8, color="#1E293B"
        )

        if square in path_set:
            ax.text(
                x + 0.5, y + 0.3, "•",
                ha="center", va="center",
                fontsize=9, color="#1D4ED8"
            )

    for start, end in SNAKES.items():
        x1, y1 = square_coordinates(start)
        x2, y2 = square_coordinates(end)

        ax.annotate(
            "",
            xy=(x2 + 0.5, y2 + 0.5),
            xytext=(x1 + 0.5, y1 + 0.5),
            arrowprops=dict(
                arrowstyle="->",
                color="#DC2626",
                lw=2,
                connectionstyle="arc3,rad=0.15"
            )
        )

    for start, end in LADDERS.items():
        x1, y1 = square_coordinates(start)
        x2, y2 = square_coordinates(end)

        ax.annotate(
            "",
            xy=(x2 + 0.5, y2 + 0.5),
            xytext=(x1 + 0.5, y1 + 0.5),
            arrowprops=dict(
                arrowstyle="->",
                color="#15803D",
                lw=2,
                connectionstyle="arc3,rad=-0.15"
            )
        )

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)
    ax.set_aspect("equal")
    ax.set_xticks([])
    ax.set_yticks([])
```

```
    ax.set_title(
        "SNAKE AND LADDER - BFS SHORTEST PATH",
        fontsize=16,
        fontweight="bold",
        pad=16
    )

    ax.text(
        0, -0.45,
        "Green: Start/Ladder | Yellow: Finish | Blue: Shortest Path | "
        "Red Arrow: Snake | Green Arrow: Ladder",
        transform=ax.transData,
        fontsize=9
    )

    plt.tight_layout()
    plt.show()


def show_results(start=1, target=100):
    path, actions, visited_count = bfs_shortest_path(start, target)

    print("=" * 55)
    print("          SNAKE AND LADDER - BFS")
    print("=" * 55)

    if path is None:
        print("No valid path exists.")
        return

    print(f"Starting square       : {start}")
    print(f"Destination square    : {target}")
    print(f"Minimum dice throws   : {len(actions)}")
    print(f"Distinct squares found: {visited_count}")

    print("\nSHORTEST PATH")
    print(" -> ".join(map(str, path)))

    print("\nMOVE-BY-MOVE DETAILS")

    for i, action in enumerate(actions, start=1):
        landing = action["landing"]
        destination = action["to"]

        message = (
            f"Move {i:02}: From {action['from']}, "
            f"roll {action['dice']}, land on {landing}"
        )

        if landing in LADDERS:
            message += f" -> Ladder up to {destination}"
        elif landing in SNAKES:
            message += f" -> Snake down to {destination}"
        else:
            message += f" -> Reach {destination}"
```

```
        print(message)

    print("\nALGORITHM")
    print("1. Start BFS from the initial square.")
    print("2. Explore all possible dice rolls.")
    print("3. Apply snakes and ladders.")
    print("4. Visit each square only once.")
    print("5. Stop when the target is reached.")
    print("6. Reconstruct the shortest path.")

    print("\nCOMPLEXITY")
    print("Time Complexity: O(V + E)")
    print("Space Complexity: O(V)")

    draw_board(path)


def run_project():
    while True:
        print("\n" + "=" * 40)
        print("       SNAKE AND LADDER - BFS")
        print("=" * 40)
        print("1. Find shortest path from 1 to 100")
        print("2. View board")
        print("3. Enter a custom starting square")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ").strip()

        if choice == "1":
            show_results()

        elif choice == "2":
            draw_board()

        elif choice == "3":
            try:
                start = int(input("Enter starting square (1-100): "))
                show_results(start, 100)
            except ValueError as error:
                print("Invalid input:", error)

        elif choice == "4":
            print("Thank you for using the BFS project!")
            break

        else:
            print("Invalid choice. Please enter a number from 1 to 4.")


run_project()
```

Fig 5.2: System code

6. RESULT

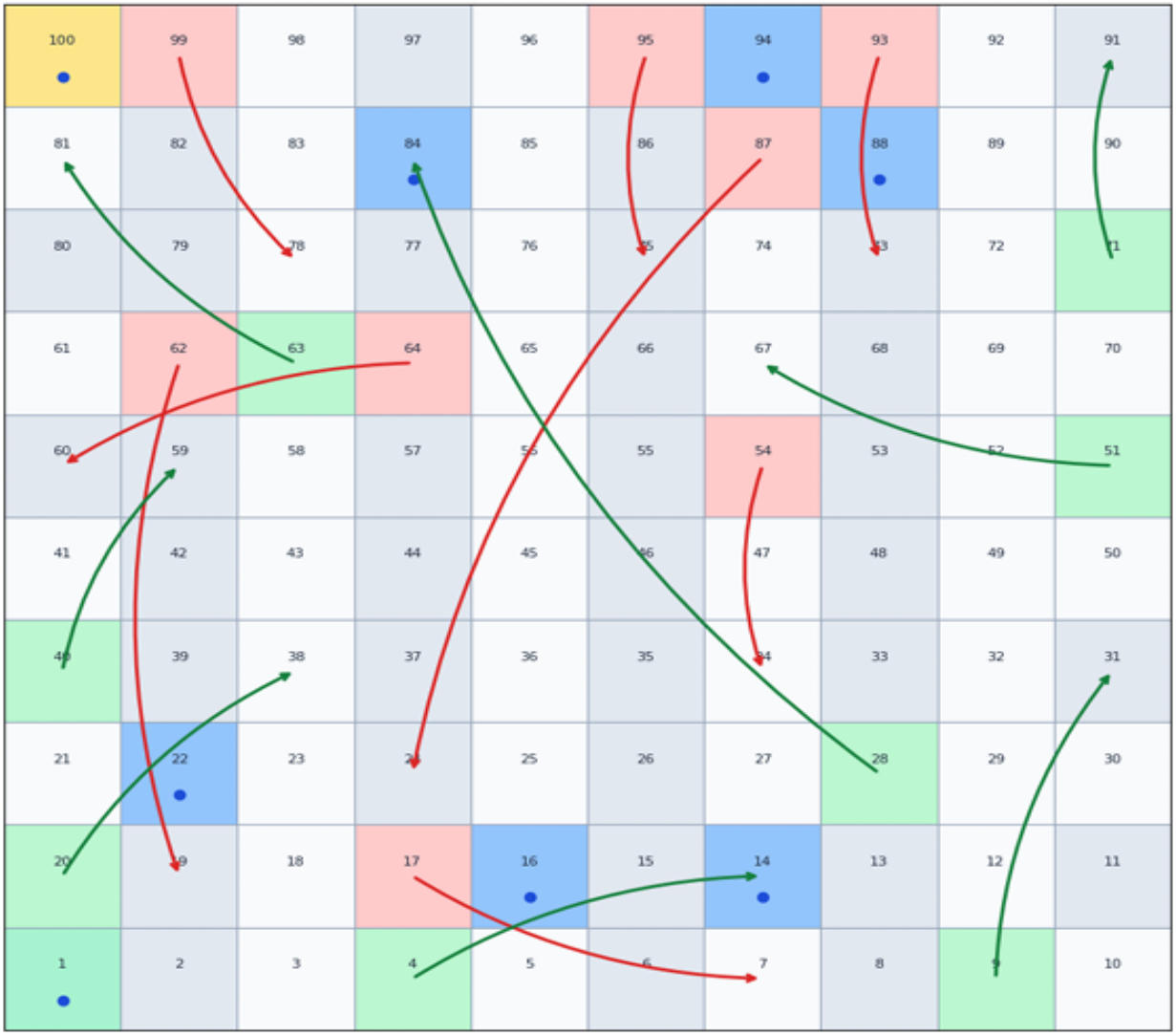
```
========================================
       SNAKE AND LADDER - BFS
========================================
1. Find shortest path from 1 to 100
2. View board
3. Enter a custom starting square
4. Exit
========================================================
         SNAKE AND LADDER - BFS
========================================================
Starting square       : 1
Destination square    : 100
Minimum dice throws   : 7
Distinct squares found: 83

SHORTEST PATH
1 -> 14 -> 16 -> 22 -> 84 -> 88 -> 94 -> 100

MOVE-BY-MOVE DETAILS
Move 01: From 1, roll 3, land on 4 -> Ladder up to 14
Move 02: From 14, roll 2, land on 16 -> Reach 16
Move 03: From 16, roll 6, land on 22 -> Reach 22
Move 04: From 22, roll 6, land on 28 -> Ladder up to 84
Move 05: From 84, roll 4, land on 88 -> Reach 88
Move 06: From 88, roll 6, land on 94 -> Reach 94
Move 07: From 94, roll 6, land on 100 -> Reach 100

ALGORITHM
1. Start BFS from the initial square.
2. Explore all possible dice rolls.
3. Apply snakes and ladders.
4. Visit each square only once.
5. Stop when the target is reached.
6. Reconstruct the shortest path.

COMPLEXITY
Time Complexity: O(V + E)
Space Complexity: O(V)
```




```
========================================
       SNAKE AND LADDER - BFS
========================================
1. Find shortest path from 1 to 100
2. View board
3. Enter a custom starting square
4. Exit
```

7. APPLICATIONS

1. **Board Game Analysis:** BFS explores the possible moves in Snake and Ladder systematically. It considers the effects of snakes and ladders to determine the minimum number of dice rolls required to reach the final square.
2. **Shortest-Path Finding**: BFS is used to find the shortest path between two points in an unweighted graph. It explores nearby nodes first and identifies the path with the minimum number of edges.
3. **Artificial Intelligence:** BFS can be used in game-solving applications to explore different possible states and find a solution. It helps analyze available moves and determine a path toward the desired goal.
4. **Network Routing:** BFS can identify a path between devices or nodes in an unweighted network. It explores connected nodes level by level to find a route with the minimum number of connections.
5. **Robotics and Path Planning:** BFS can help robots navigate grids or simple environments by finding a path from a starting position to a target while avoiding blocked locations.
6. **Educational Purposes**: This project helps students understand graph representation, queue operations, visited-node tracking, and shortest-path algorithms through a familiar board game.

8. FUTURE WORK

The proposed Snake and Ladder game using the Breadth-First Search (BFS) algorithm can be enhanced in the future by introducing additional features to improve its functionality and user experience.

1. **Graphical User Interface (GUI):** Develop an interactive interface with a visual board, dice, snakes, and ladders.
2. **Multiplayer Support:** Allow multiple players to participate in the game.
3. **Different Board Configurations:** Support boards of different sizes and different snake and ladder positions.
4. **Algorithm Comparison:** Compare BFS with other algorithms to analyze their performance in solving the game.
5. **Path Visualization:** Display the shortest path identified by BFS on the game board.
6. **Advanced Game Features:** Add score tracking, game statistics, and restart options.

   These enhancements can make the game more interactive and demonstrate the broader applications of graph traversal and shortest-path algorithms.

9. CONCLUSION

The mini-project, “Implementation of Snake and Ladder Using Breadth-First Search (BFS),” demonstrates the practical application of graph traversal algorithms in solving a traditional board game. The board is represented as a graph, where each square represents a vertex and possible dice moves represent edges. Snakes and ladders are considered while determining the next position.

The BFS algorithm explores possible moves level by level using a queue and helps determine the minimum number of dice rolls required to reach the destination. Through this project, the concepts of graph representation, queue operations, visited-node tracking, and shortest-path algorithms are understood more clearly.

This project improves problem-solving skills and demonstrates how BFS can be applied to real-world search problems. In the future, the system can be enhanced with a graphical user interface, multiplayer support, and shortest-path visualization. Overall, the project provides a simple and effective demonstration of BFS and its role in solving shortest-path problems.

10. REFERENCE

[1] GeeksforGeeks. “Snake and Ladder Problem.” Available at: https://www.geeksforgeeks.org/dsa/snake-ladder-problem-2/

[2] CP-Algorithms. “Breadth First Search.” Available at: https://cp-algorithms.com/graph/breadth-first-search.html

[3] LeetCode. “909. Snakes and Ladders.” Available at: https://leetcode.com/problems/snakes-and-ladders/

[4] Boost C++ Libraries. “Breadth-First Search.” Available at: https://www.boost.org/doc/libs/latest/libs/graph/doc/html/graph/algorithms/traversal/breadth_first_search.html

[5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 4th Edition. MIT Press, 2022. Available at: https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[6] Joel John. “Shortest Path Predictor for the Snakes and Ladder Problem Using Breadth-First Search Algorithm.” GitHub repository. Available at: https://github.com/joeljohn135/Shortest-Path-Predictor-for-the-Snakes-and-ladder-Problem-Using-Breadth-First-Search-Algo